

# Network Monitoring Systems & Security Automation

**IDS/IPS (Snort & Suricata) + ML Integration + WebSocket Realtime Dashboard**

**Duration:** 120 minutes

**Instructor:** Assist. Prof. Dr. Chanankorn Jandaeng

Walailak University

 [chatchanan.ja@mail.wu.ac.th](mailto:chatchanan.ja@mail.wu.ac.th)

# 1. Network Monitoring Systems

**Goal:** Observe, analyze, and protect network communications.

**Why?**

- Identify threats, anomalies, or policy violations.
- Provide visibility across infrastructure.
- Support forensics and compliance.

# 1. Network Monitoring Systems

## Monitoring Layers:

User Behavior → Application Layer → Network Packets → Physical Links

## Key Tools:

- **Wireshark:** Packet analysis
- **Snort/Suricata:** IDS/IPS detection
- **Zeek:** Behavioral analysis
- **SIEM (ELK/Wazuh):** Centralized correlation

## 2. IDS/IPS Fundamentals

### Definition

| Type | Description                                                |
|------|------------------------------------------------------------|
| IDS  | Intrusion Detection System — passive detection & alerting  |
| IPS  | Intrusion Prevention System — inline blocking & mitigation |

### Function:

Monitor network traffic → Apply rules/patterns → Detect or block malicious activity.

## IDS vs IPS Comparison

| Feature | IDS                     | IPS                         |
|---------|-------------------------|-----------------------------|
| Mode    | Passive (Tap/SPAN)      | Inline (Firewall gateway)   |
| Action  | Alert only              | Drop/block packets          |
| Risk    | Zero network disruption | May impact latency          |
| Example | Suricata IDS mode       | Suricata + NFQUEUE (inline) |

# Detection Techniques

| Technique       | Concept                 | Strength                   | Limitation             |
|-----------------|-------------------------|----------------------------|------------------------|
| Signature-based | Match known patterns    | Accurate for known threats | Misses zero-days       |
| Anomaly-based   | Detect unusual behavior | Detects new attacks        | Higher false positives |
| Hybrid          | Combine both            | Balanced                   | Complex tuning         |

### 3. Core IDS/IPS Components

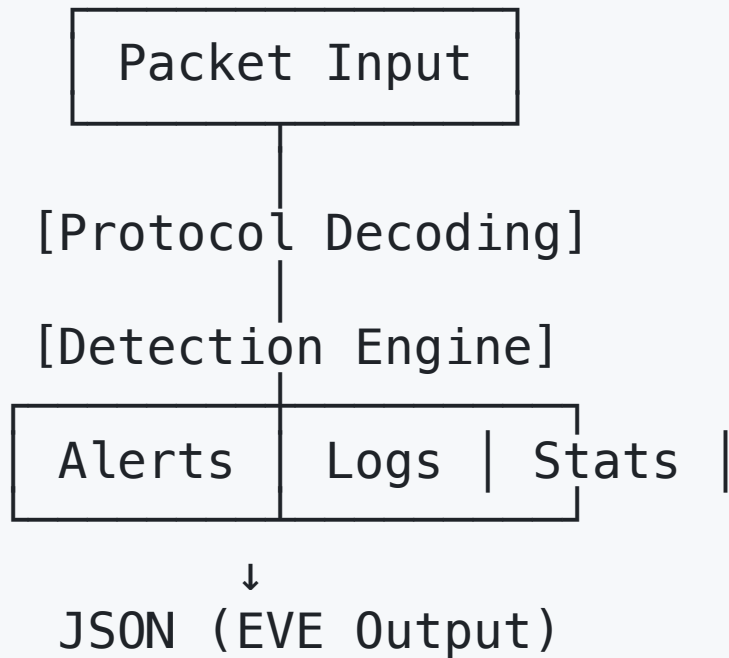
| Component           | Description                                     |
|---------------------|-------------------------------------------------|
| Packet Capture      | Raw network data (libpcap, AF_PACKET)           |
| Decoder             | Protocol parsing (Ethernet, IP, TCP, DNS, HTTP) |
| Detection Engine    | Rule and pattern matching                       |
| Logger              | Output alerts (EVE JSON, Syslog)                |
| Console / Dashboard | Management and visualization                    |

# Snort vs Suricata

| Feature        | Snort           | Suricata                   |
|----------------|-----------------|----------------------------|
| Developer      | Cisco           | OISF                       |
| Multithreading | No              | Yes                        |
| Rule Format    | Snort rules     | Compatible (Snort-style)   |
| Output         | Text            | JSON (EVE)                 |
| Performance    | Single core     | Multi-core optimized       |
| Use Case       | Legacy networks | Modern high-speed networks |



## 4. Suricata Architecture



## Suricata Rule Example

```
alert tcp any any -> any 80 (msg:"HTTP Shell Upload"; content:"cmd="; nocase; sid:10001;)
```

### Fields:

- `alert` = action
- `tcp any any -> any 80` = match criteria
- `(msg:"HTTP Shell Upload"; content:"cmd="; sid:10001;)` = options

✓ **Snort-compatible syntax**

## Example Rule — ICMP Flood

```
alert icmp any any -> 192.168.1.10 any (msg:"ICMP Flood Detected"; itype:8; threshold:10 both, track by_dst, count 50, seconds 1; sid:300001;)
```

Detects ICMP ping floods (>50 packets/sec).

## 5. Alert Handling

### EVE JSON Format

```
{  
  "timestamp": "2025-11-06T09:44:02.100Z",  
  "src_ip": "192.168.1.20",  
  "dest_ip": "10.0.0.8",  
  "proto": "TCP",  
  "alert": {  
    "signature_id": 10001,  
    "signature": "HTTP Shell Upload",  
    "severity": 3  
  }  
}
```

## 5. Alert Handling

### Alert Response Steps

1. **Detect** → rule triggered
2. **Validate** → check context
3. **Respond** → block, isolate, or notify
4. **Tune** → modify rules to reduce false positives

## 6. Demonstration: Suricata in Action

### Environment

- OS: Ubuntu / Kali Linux
- IDS: Suricata
- Attack Tool: hping3, nmap, Hydra
- Log Viewer: `jq`, Kibana, or Python parser

## Example: Simulate Ping Flood

```
sudo hping3 --icmp -d 1200 -i u100 192.168.1.10
```

✓ Suricata rule triggers flood alert.

## Example: Detect HTTP Brute Force

```
hydra -l admin -P pass.txt http-post-form "login.php:user=^USER^&pass=^PASS^:F=Invalid"
```

### Rule:

```
alert http any any -> any any (msg:"Brute Force"; flow:to_server,established; content:"login"; detection_filter:track by_src, count 10, seconds 5; sid:300002;)
```



## Check Alerts

```
sudo tail -f /var/log/suricata/eve.json | jq '.'
```

or integrate with ELK:

```
Filebeat → Logstash → Elasticsearch → Kibana
```

## 7. Security Automation and Integration

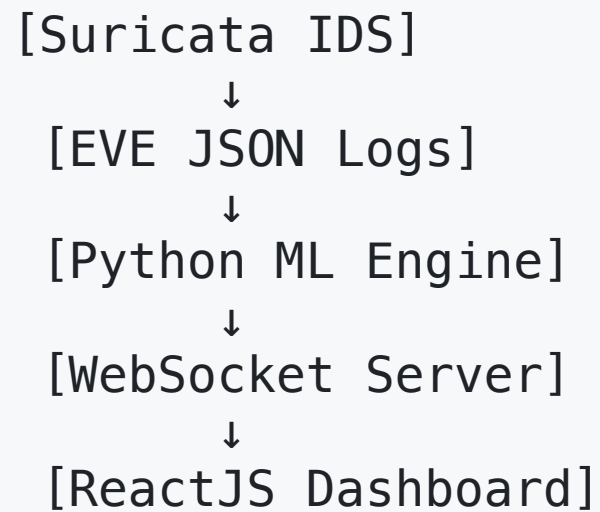
### Problem:

Traditional IDS produces thousands of alerts → manual review impossible.

### Solution:

Integrate **Machine Learning (ML)** for automatic correlation, severity scoring, and anomaly detection.

## Architecture Overview



```
graph TD; A[Suricata IDS] --> B[EVE JSON Logs]; B --> C[Python ML Engine]; C --> D[WebSocket Server]; D --> E[ReactJS Dashboard];
```

[Suricata IDS]  
↓  
[EVE JSON Logs]  
↓  
[Python ML Engine]  
↓  
[WebSocket Server]  
↓  
[ReactJS Dashboard]

## 8. Machine Learning Integration

### Workflow

1. Extract Suricata logs (EVE JSON).
2. Convert to numerical features.
3. Train ML model (e.g., Isolation Forest).
4. Classify alerts as **normal/anomalous**.
5. Stream results to dashboard in real-time.

## Example ML Integration Code

```
import json, pandas as pd
from sklearn.ensemble import IsolationForest

with open("eve.json") as f:
    alerts = [json.loads(l) for l in f if '"alert"' in l]

df = pd.DataFrame([
    {"sig_id": a["alert"]["signature_id"],
     "severity": a["alert"]["severity"]}
    for a in alerts])

model = IsolationForest(contamination=0.1)
df["score"] = model.fit_predict(df)
df["label"] = df["score"].map({1:"normal", -1:"anomaly"})
print(df.head())
```

✓ Model isolates rare / suspicious alerts.

## 9. Real-time Visualization with WebSocket

### Why WebSocket?

- Persistent two-way connection.
- Push new IDS alerts immediately.
- Ideal for SOC dashboards and live incident response.

## FastAPI Backend with WebSocket

```
from fastapi import FastAPI, WebSocket
import json, asyncio

app = FastAPI()

@app.websocket("/ws/alerts")
async def ws_alerts(ws: WebSocket):
    await ws.accept()
    while True:
        with open("/var/log/suricata/eve.json") as f:
            data = [json.loads(line) for line in f.readlines()[-5:]]
        await ws.send_text(json.dumps(data))
        await asyncio.sleep(2)
```

✓ Sends last 5 alerts every 2 seconds.

## ReactJS Frontend Example

```
import React, { useEffect, useState } from "react";

export default function LiveAlerts() {
  const [alerts, setAlerts] = useState([]);
  useEffect(() => {
    const ws = new WebSocket("ws://localhost:8000/ws/alerts");
    ws.onmessage = e => setAlerts(JSON.parse(e.data));
    return () => ws.close();
  }, []);
  return (
    <div className="p-6">
      <h2 className="text-2xl font-bold mb-2">Live Suricata Alerts</h2>
      <ul>
        {alerts.map((a, i) => (
          <li key={i} className="border-b py-1">
            [{a.timestamp}] {a.alert.signature} ({a.src_ip} → {a.dest_ip})
          </li>
        ))}
      </ul>
    </div>
  );
}
```



## 10. Full Automation Flow

```
[Suricata → EVE JSON]
      ↓
[Python ML Engine] → Classify severity
      ↓
[FastAPI WebSocket] → Stream live data
      ↓
[ReactJS Dashboard] → Display anomalies instantly
```

### Outcome:

A prototype of **Intelligent IDS** with automated alerting and real-time SOC visualization.

## 11. Demonstration Plan (60 min Lab)

| Step | Activity                                 | Tool            |
|------|------------------------------------------|-----------------|
| 1    | Setup Suricata in IDS mode               | Suricata        |
| 2    | Simulate attack (ICMP, HTTP brute-force) | hping3 / Hydra  |
| 3    | Parse EVE logs with Python               | Pandas          |
| 4    | Train ML model (Isolation Forest)        | scikit-learn    |
| 5    | Deploy WebSocket + ReactJS dashboard     | FastAPI / React |
| 6    | Observe live anomaly feed                | Browser         |

## 12. Summary

| Concept               | Description                                            |
|-----------------------|--------------------------------------------------------|
| <b>IDS/IPS</b>        | Detects or blocks malicious network traffic            |
| <b>Suricata</b>       | High-performance, open-source IDS with EVE JSON output |
| <b>Alert Handling</b> | From detection → validation → response                 |
| <b>Automation</b>     | ML-based anomaly scoring and classification            |
| <b>Visualization</b>  | WebSocket-powered realtime dashboards                  |
| <b>Integration</b>    | Forms the foundation of an intelligent SOC pipeline    |

# End of Lecture

**Assist. Prof. Dr. Chanankorn Jandaeng**

Walailak University

 [chatchanan.ja@mail.wu.ac.th](mailto:chatchanan.ja@mail.wu.ac.th)



# Lecture Highlights

| Area                | Learning Outcome                                             |
|---------------------|--------------------------------------------------------------|
| Theory              | Understood IDS/IPS architecture, rules, and alert management |
| Practice            | Installed and configured Suricata, simulated attacks         |
| Automation          | Linked ML model to Suricata for intelligent detection        |
| Realtime Monitoring | Built WebSocket + ReactJS dashboard                          |
| Research Insight    | Foundation for AI-assisted intrusion detection systems       |